HW2

Kylie Dickinson

2026-04-30

I. Approach of Metadata

I-1. General Metadata Format

The metadata follows a format structure of:

`date-time logging-host app PID message`

For my approach, I first wanted to ignore any miscellaneous lines, including blank lines and header lines (for the log files). Before removing, the total number of lines in the file is found to be 99,968.

```
keep = !grepl("^#", lines) & !grepl("^\\s*$", lines)
```

I wrote a general pattern that captures any abbreviated month with its date and time, any letter/numerical string that contains periods, underscores, and dashes for a logging host, and a catch-all for the message. The app/PID required multiple rounds of testing, ultimately landing on the same expression as the logging host, with an optional second term for the app if it's two words, and an optional identifier for a PID.

```
pattern = paste0(
  "^([A-Z][a-z]{2}\\s{1,2}\\d{1,2}\\s+\\d{2}:\\d{2}:\\d{2})",
  "\\s+",
  "([A-Za-z0-9._-]+)",
  "\\s+",
  "([A-Za-z0-9\\._-]+)(\\s[A-Za-z0-9._-]+)?(\\[\\d+\\])?",
  ":\\s",
  "(.+)$"
)
```

It's found that there are now 99,960 lines, with 99,053 lines matching this general pattern and 907 not matching. Before moving on to special exceptions, a log labels vector is created for the dataframe so that each line will have its respective log file. The dataframe consists of six columns and one temporary column called `matched`, using a written function called `pull_col` in tangent with `regexec` and `regmatches`:

- date_time
- logging_host
- app
- pid
- message
- log_file
- matched

```
pull_col = function(list, index) {
  sapply(list, function(x) if (length(x) == 0) NA_character_ else x[index])
}
```

The `matched` column indicates which lines in the dataframe are filled. Since `regmatches` fills in `character(0)` for non-matches, `FALSE` values will be additionally evaluated as special pattern exceptions.

I-2. Special Metadata Exceptions

An exploration of the non-matching data shows four case exceptions to the general pattern, all having to do with app/PID formats. Case 1 addresses the format of **app(module)[pid]**, case 2 addresses the format of **app[number part of app] ([pid])**, case 3 addresses the format of **com.apple.xpc.launchd[1] (com.apple. . . WebContent.37963)**, and case 4 addresses the format of **com.apple.xpc.launchd[1] (com.apple.WebKit.Networking.A546008E-. . . [33438])**. The counts of these cases are 853, 35, 18, and 1, which adds up to 907 (our total number of non-matches).

Once each case is pattern matched, they are cleaned to fit the format of the general metadata and filled back into the original dataframe.

The `matched` column is now removed after realizing 99,960 matches, and the dataframe `df` is complete.

Table 1: Log DataFrame Preview (first 10 rows)

date_time	logging_host	app	pid	message	log_file
Nov 30 06:39:00	ip-172-31-27-153	CRON	21882	`pam_unix(cron:session): session closed for user root`	auth.log
Nov 30 06:47:01	ip-172-31-27-153	CRON	22087	`pam_unix(cron:session): session opened for user root by (uid=0)`	auth.log
Nov 30 06:47:03	ip-172-31-27-153	CRON	22087	`pam_unix(cron:session): session closed for user root`	auth.log
Nov 30 07:07:14	ip-172-31-27-153	sshd	22116	`Connection closed by 122.225.103.87 [preauth]`	auth.log
Nov 30 07:07:35	ip-172-31-27-153	sshd	22118	`Connection closed by 122.225.103.87 [preauth]`	auth.log
Nov 30 07:08:13	ip-172-31-27-153	sshd	22120	`Connection closed by 122.225.103.87 [preauth]`	auth.log
Nov 30 07:17:01	ip-172-31-27-153	CRON	22125	`pam_unix(cron:session): session opened for user root by (uid=0)`	auth.log
Nov 30 07:17:01	ip-172-31-27-153	CRON	22125	`pam_unix(cron:session): session closed for user root`	auth.log
Nov 30 08:17:01	ip-172-31-27-153	CRON	22172	`pam_unix(cron:session): session opened for user root by (uid=0)`	auth.log
Nov 30 08:17:01	ip-172-31-27-153	CRON	22172	`pam_unix(cron:session): session closed for user root`	auth.log

II. Data Validation

II-1. Log Files

To formally begin data validation and exploration, we'll take a look at the log files. There are 5 log files in this specific order:

- auth.log
- auth2.log
- loghub/Linux/Linux_2k.log
- loghub/Mac/Mac_2k.log
- loghub/OpenSSH/SSH_2k.log

They make up 86,839, 7121, 2000, 2000, and 2000 lines respectively. To dive further into the individual logs, it's worth looking at their dates. If the date range of the entire file is dependent on the order of the logs, then it would start Nov 30 06:39:00 in **auth.log**, and end in Dec 10 11:04:45 in the **SSH_2k.log**. Each log has the following date range:

auth.log: Nov 30 06:39:00 – Dec 31 22:27:48 (31.65889 days)

auth2.log: Mar 27 13:06:56 – Apr 20 14:14:29 (24.04691 days)

loghub/Linux/Linux_2k.log: Jun 14 15:16:01 – Jul 27 14:42:00 (42.97638 days)

loghub/Mac/Mac_2k.log: Jul 01 09:00:55 – Jul 08 08:10:46 (6.965174 days)

loghub/OpenSSH/SSH_2k.log: Dec 10 06:55:46 – Dec 10 11:04:45 (0.1729051 days)

Since these dates do not include the year, for additional context, it was attempted to try and comb through the messages in the dataframe to see if any information could be extracted. UNIX was launched in 1971, so as an assurance, the years 1971–2026 were used in `regex` pattern matching in the messages. A function was written called `explore_year` to analyze each individual log file and expression search for an inputted year. First, the unique "years" were found in the messages, and using `explore_year`, the context of the message was taken into account. Some "years" were disqualified due to not actually being a year. It was found that there were two relevant years in these 99,960 message lines: **2005** for the Linux log, and **2017** for the Mac log. This indicates the lines are not completely linear in time and are independent between logs.

Below is the method of year grabbing, as well as year testing:

```
year_pattern = "\\b(19[7-9][0-9]|200[0-9]|201[0-9]|202[0-6])\\b" # 1971-2026

for (log in unique(df$log_file)) {
  log_df = df[df$log_file == log, ]

  has_year = !is.na(log_df$message) & grepl(year_pattern, log_df$message)
  year_rows = log_df[has_year, ]

  cat("\n", rep("=", 50), "\n", sep="")
  cat("Log file:", log, "\n")
  cat("Rows with year in message:", nrow(year_rows), "\n")

  if (nrow(year_rows) > 0) {
    years_found = regmatches(year_rows$message, gregexpr(year_pattern, year_rows$message))
    years_flat = unlist(years_found)

    cat("Unique years found:", paste(sort(unique(years_flat)), collapse=", "), "\n\n")

    sample_rows = head(year_rows, 10)
    for (i in seq_len(nrow(sample_rows))) {
      cat(sprintf("  [%s] app=%-20s msg=%s\n",
                  sample_rows$date_time[i],
                  sample_rows$app[i],
                  sample_rows$message[i]))
    }
  } else {
    cat("  (No year mentions found in messages)\n")
  }
}

explore_year = function(year, log_file_name) {
  year_pattern = paste0("\\b", year, "\\b")
```

```
  log_df = df[df$log_file == log_file_name, ]

  has_year = !is.na(log_df$message) & grepl(year_pattern, log_df$message)
  year_rows = log_df[has_year, ]

  cat(rep("=", 60), "\n", sep="")
  cat("Year:", year, "| Log file:", log_file_name, "\n")
  cat("Matching rows:", nrow(year_rows), "\n")
  cat(rep("=", 60), "\n", sep="")

  if (nrow(year_rows) == 0) {
    cat("No messages containing", year, "in this log file.\n")
    return(invisible(NULL))
  }

  for (i in seq_len(nrow(year_rows))) {
    cat(sprintf("\n[%d] %s | app: %s\n", i, year_rows$date_time[i], year_rows$app[i]))
    cat("    ", year_rows$message[i], "\n")
  }

  return(invisible(year_rows))
}
```

Since the years were captured for the Linux and Mac logs, their time-ranges can be seen in the following two bar plots:

Figure 1: Mac Log − Activity Over Time (2017)

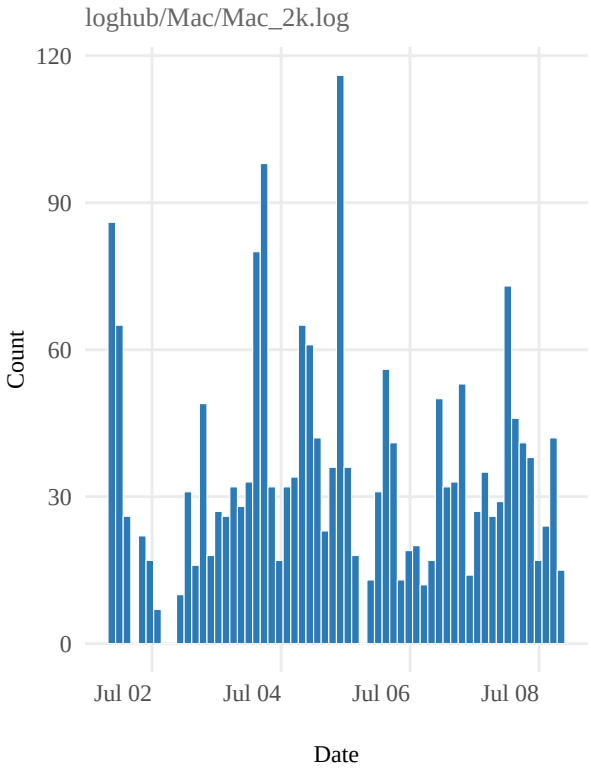
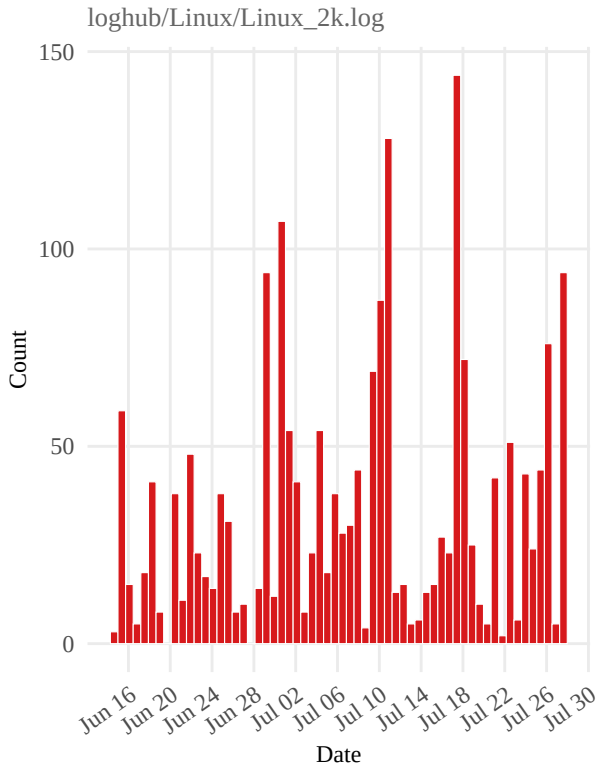


Figure 2: Linux Log − Activity Over Time (2



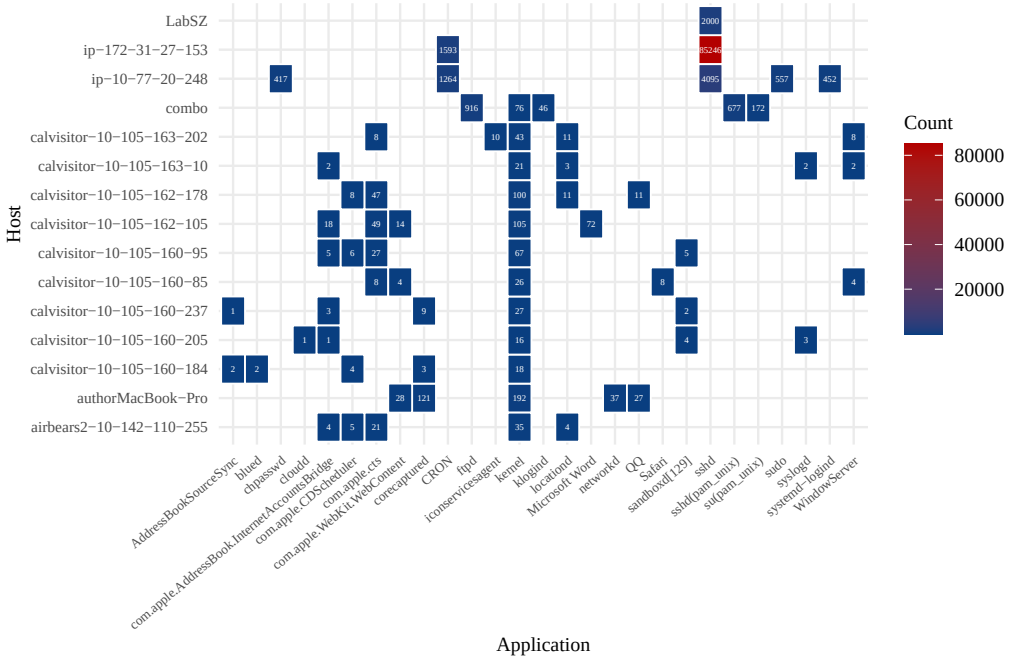
II-1. Apps, PIDs, & Logging Hosts

Moving on from the date-time, for PIDs, it's verified that they are all numbers before they are converted to an integer column. With the apps, only 4 out of the 104 contain numbers (e.g., `syslogd 1.4.1`, `sandboxd[129]`, `com.apple.xpc.launchd[1]`, `BezelServices 255.10`), where **syslogd 1.4.1** and **BezelServices 22.10** contain versions. The others have an additional structure. The most common apps are shown in Table 2, and Figure 3 displays the top 15 hosts and the 5 highest apps logging information from that host.

Table 2: Top 20 Applications by Count

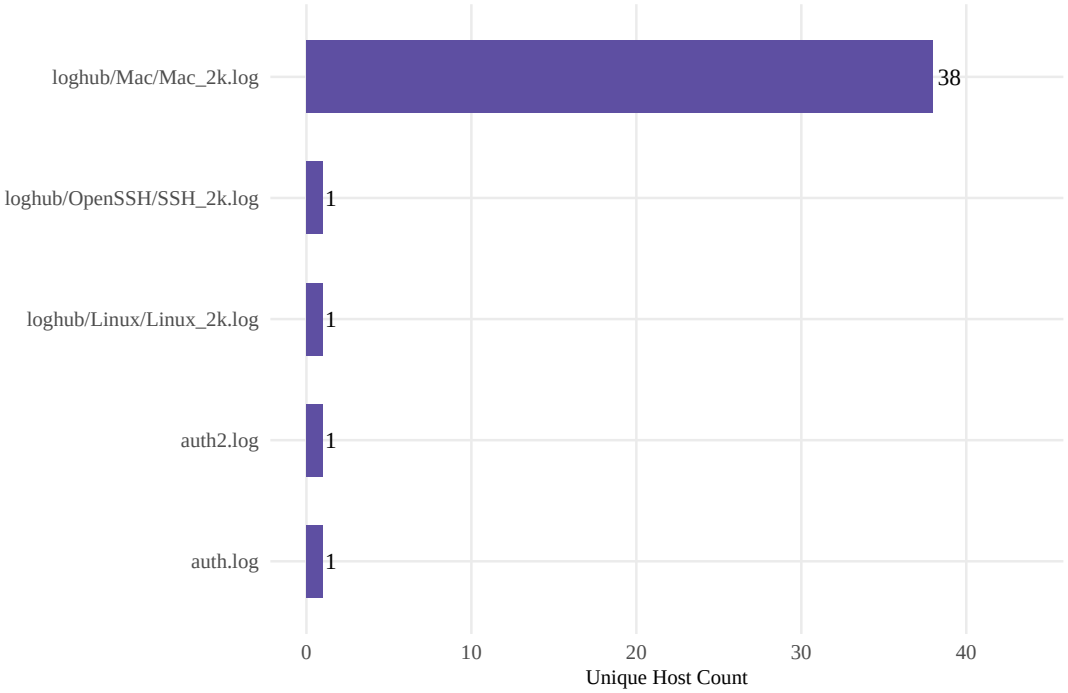
Application	Count
sshd	91341
CRON	2857
ftpd	916
kernel	851
sshd(pam_unix)	677
sudo	557
systemd-logind	452
chpasswd	417
systemd	238
su(pam_unix)	172
com.apple.cts	166
corecaptured	158
QQ	75
Microsoft Word	72
com.apple.WebKit.WebContent	60
locationd	60
com.apple.AddressBook.InternetAccountsBridge	59
useradd	50
klogind	46
su	45

Figure 3: Top 5 Apps per Logging Host (top 15 hosts)



There is only one logging host for almost all log files except **loghub/Mac/Mac_2k.log**, which has 38 different logging hosts.

Figure 4: Unique Logging Hosts per Log File



There are 37 logging hosts that resemble IP addresses, seen in Figure 5.

Table 3: IP-like Logging Hosts Count

Host	Count
ip-172-31-27-153	86839
ip-10-77-20-248	7121
calvisitor-10-105-162-105	338
calvisitor-10-105-162-178	256
calvisitor-10-105-160-95	140
calvisitor-10-105-163-202	137
calvisitor-10-105-160-85	83
calvisitor-10-105-160-237	53
calvisitor-10-105-160-184	39
calvisitor-10-105-163-10	34
calvisitor-10-105-160-205	30
calvisitor-10-105-162-124	29
calvisitor-10-105-162-32	27
calvisitor-10-105-163-253	26
calvisitor-10-105-160-179	19
calvisitor-10-105-160-226	17
calvisitor-10-105-161-225	16
calvisitor-10-105-162-107	13
calvisitor-10-105-160-37	12
calvisitor-10-105-160-210	9
calvisitor-10-105-162-98	8
calvisitor-10-105-160-22	7
calvisitor-10-105-160-181	6

Table 3: IP-like Logging Hosts Count *(continued)*

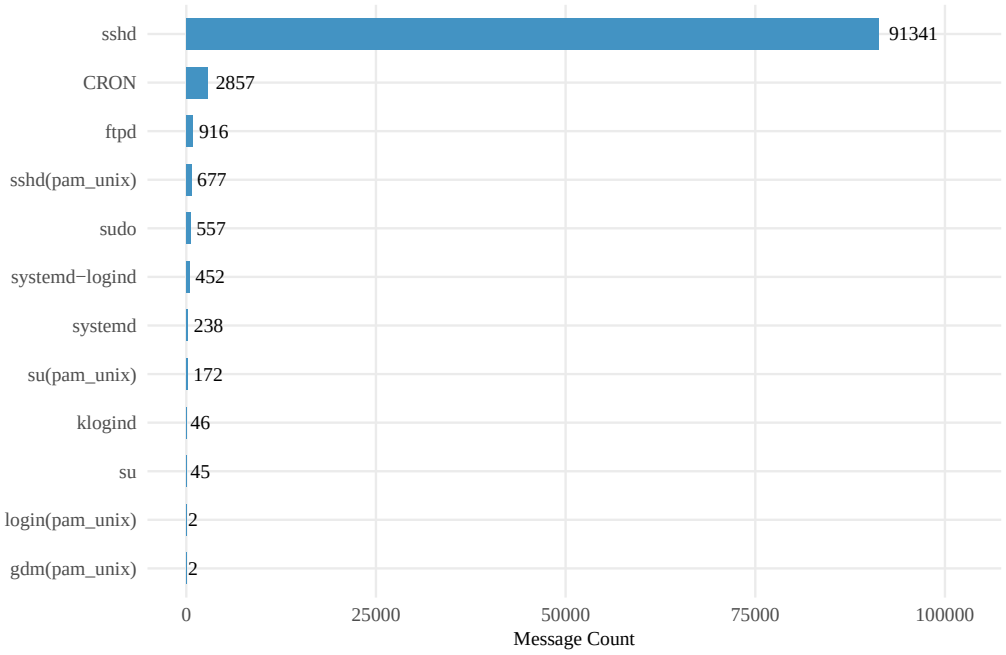
Host	Count
calvisitor-10-105-160-47	6
calvisitor-10-105-161-176	6
calvisitor-10-105-162-228	5
calvisitor-10-105-163-147	5
calvisitor-10-105-162-108	4
calvisitor-10-105-162-175	4
calvisitor-10-105-163-168	4
calvisitor-10-105-163-9	4
calvisitor-10-105-162-138	3
calvisitor-10-105-162-211	3
calvisitor-10-105-162-81	3
calvisitor-10-105-161-231	2
calvisitor-10-105-161-77	2
calvisitor-10-105-163-28	2

III. Approach of Logins

III-1. Successful Logins

To find successful logins, keywords are required to be able to go through the 99,960 lines. My strategy was to familiarize myself with the 104 different applications, finding which are more relevant to login message information. This was done by doing a small preview of each app with some message lines. After an initial run through, I denoted the relevant/important apps and previewed an even larger portion of messages, fine tuning any keywords for successful (and invalid) logins. The apps I focused on were: `CRON`, `sshd`, `systemd-logind`, `systemd`, `sudo`, `su`, `sshd(pam_unix)`, `su(pam_unix)`, `ftpd`, `klogind`, `login(pam_unix)`, and `gdm(pam_unix)`.

Figure 5: Message Counts for Login−Related Applications



I also wrote a function called `show_all_messages` to get a greater understanding of what pattern the lines follow for the important apps.

```
show_all_messages = function(app_name) {
  index = df$app == app_name
  count = sum(index)
  cat("\nApp:", app_name, "(", count, "total )\n")
  cat(rep("-", 40), "\n", sep="")
  print(df$message[index])
}
```

The keywords grabbed for successful logins were:

"Accepted"

"session opened"

"connection from"

"New session"

This totaled 3550 lines out of 99,960. After pattern matching and creating a separate dataframe for successful logins, Table 4 shows a preview of 20 users and their associated IPs.

Table 4: Users and IPs of Successful Logins

App	User	IP
sshd	ubuntu	85.245.107.41
sshd	ubuntu	85.245.107.41
sshd	ubuntu	85.245.107.41
sshd	ubuntu	85.245.107.41
sshd	ubuntu	85.245.107.41
sshd	ubuntu	85.245.107.41
sshd	ubuntu	85.245.107.41
sshd	ubuntu	85.245.107.41
sshd	ubuntu	85.245.107.41
sshd	ubuntu	85.245.107.41
sshd	ubuntu	85.245.107.41
sshd	ubuntu	85.245.107.41
sshd	ubuntu	85.245.107.41
sshd	ubuntu	85.245.107.41
sshd	ubuntu	85.245.107.41
sshd	elastic_user_7	127.0.0.1
sshd	ubuntu	85.245.107.41
sshd	elastic_user_2	85.245.107.41
sshd	elastic_user_8	85.245.107.41

As an additional way to preserve information, for `su`, `sudo`, and `login(pam_unix)`, the issuing user was preserved in a separate column in the successful login dataframe for a total of 196 lines.

Table 5: Successful Logins with Issuing User

App	User	Issuing User	IP
sudo	root	ubuntu	NA
sudo	root	ubuntu	NA

Table 5: Successful Logins with Issuing User *(continued)*

App	User	Issuing User	IP
sudo	root	ubuntu	NA
sudo	root	ubuntu	NA
sudo	root	ubuntu	NA
sudo	root	ubuntu	NA
sudo	root	ubuntu	NA
sudo	root	ubuntu	NA
sudo	root	ubuntu	NA
sudo	root	ubuntu	NA
sudo	root	ubuntu	NA

III-2. Invalid Logins

The same strategy was applied to invalid logins, with the following keywords:

"Invalid user"

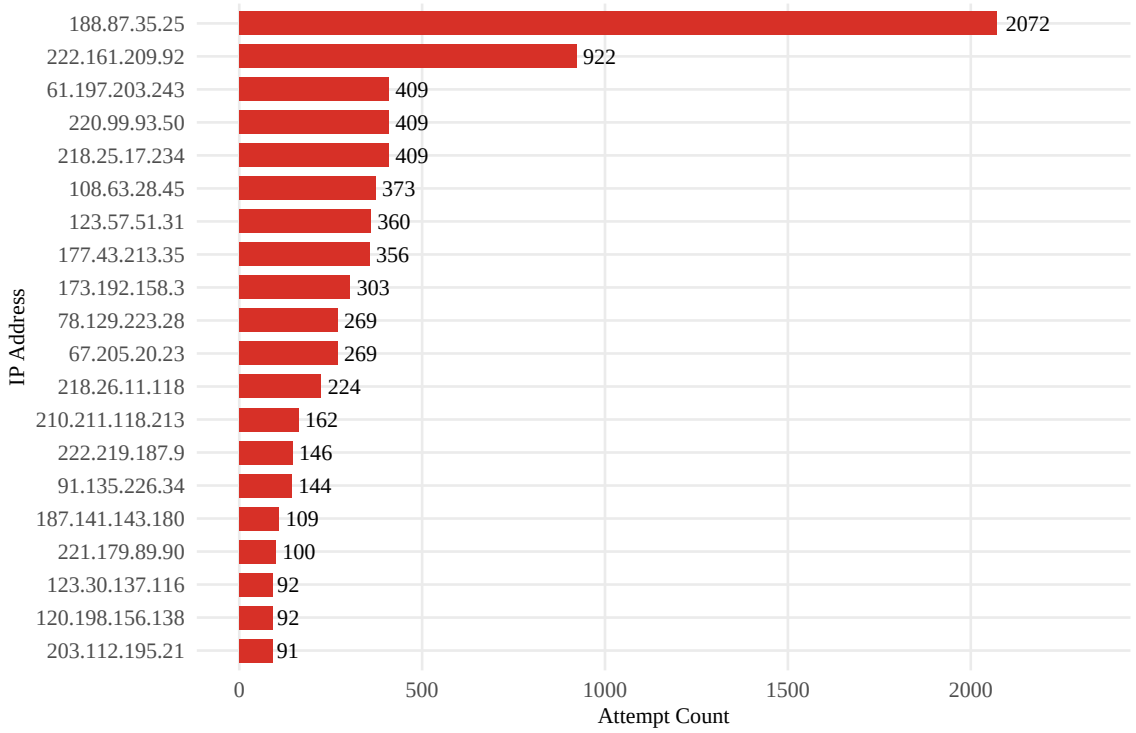
"authentication failure"

"[Aa]uthentication failed"

"POSSIBLE BREAK-IN ATTEMPT"

This totaled 19113 lines out of 99,960. After pattern matching and creating a separate dataframe for invalid logins, a preview of the top 20 invalid IP addresses are shown in Figure 6.

Figure 6: Top 20 IPs by Invalid Login Attempts



Cross-referencing these invalid login related IP addresses to IP addresses in valid logins produces one address **85.245.107.41** that appears in both types of logins:

Table 6: IPs in Valid and Invalid Logins

IP	invalid_attempts	successful_logins
85.245.107.41	2	174

There were also multiple IP addresses that used the same invalid user login, seen in Table 7. The column **Total Unique IPs** denotes the total number of IPs detected per that user. The column **Shared Network Count** denotes there are x number of IP addresses amongst those **Total Unique IPs** that have the same network and domain. There are a total of 36 users that ran into IP addresses with the same network/domain, and 116 users who did not have this problem.

```
shared_netdom_count = 0
for (user in names(multiple_ip_users)) {
  ips = unique(invalid_user_ip$ip[invalid_user_ip$user == user]) # gets unique IPs per user
  ip_network_domain = sub("^(\\d+\\.\\d+\\.\\d+)\\..*", "\\1", ips)  # e.g., 218.188.2
  if (any(duplicated(ip_network_domain))) {
    shared_netdom_count = shared_netdom_count + 1
    cat("\nUser:", user, "\n")
    cat("Shared domain/network:",
        paste(unique(ip_network_domain[duplicated(ip_network_domain)]), collapse=", "), "\n")
  }
}
cat("Users with IPs from same network/domain:", shared_netdom_count, "\n")
cat("Users with IPs from different network/domains:",
    length(multiple_ip_users) - shared_netdom_count, "\n")
```

Table 7: Users with IPs Sharing Network Domains

User	Total Unique IPs	Shared Network Count
admin	825	44
debug	267	13
vyatta	223	12
log	260	10
karaf	225	10
ftpuser	323	9
D-Link	280	8
PlcmSpIp	252	8
guest	230	8
default	215	8
xbian	207	8
dreamer	205	8
pi	219	7
xbmc	184	7
support	175	7
arbab	200	6
ubnt	186	6
oracle	179	6
test	189	5
agata	160	5
ftp	159	5

Table 7: Users with IPs Sharing Network Domains *(continued)*

User	Total Unique IPs	Shared Network Count
bill	154	5
user	154	5
info	150	5
mike	144	5
marketing	142	5
uploader	128	5
git	16	1
hadoop	7	1
redmine	5	1
upload	4	1
hduser	3	1
ibmuser	3	1
manager	3	1
rsync	3	1
ooooooooooooooo	2	1

III-3. Authentication Failures

There have also been authentication failures, with the keywords used being "authentication", "authenticating", and "authenticate". There were 4540 lines related to authentication failures. The relevant apps are `sshd`, `sshd(pam_unix)`, `klogind`, `gdm(pam_unix)`, `gdm-binary`, and `UserEventAgent`.

```
auth_index = grepl("authentication|authenticating|authenticate", df$message, ignore.case=TRUE)
sum(auth_index, na.rm=TRUE)
unique(df$app[auth_index])

# for (app in unique(df$app[auth_index])) {
#   cat("\nApp:", app, "\n")
#   cat(rep("-", 40), "\n", sep="")
#   print(head(df$message[df$app == app & auth_index], 50))
# }
auth_index = grepl("authentication|authenticating|authenticate", df$message, ignore.case = TRUE) &
             df$app %in% c("klogind", "sshd(pam_unix)", "sshd")
auth_fail_df = df[auth_index, ]
auth_fail_df$ip = NA_character_

  # klogind: "Authentication failed from 163.27.187.39"
index = auth_fail_df$app == "klogind"
m = regexec("from ([0-9.]+)", auth_fail_df$message[index])
p = regmatches(auth_fail_df$message[index], m)
auth_fail_df$ip[index] = pull_col(p, 2)

  # sshd(pam_unix): rhost= format
index = auth_fail_df$app == "sshd(pam_unix)"
m = regexec("rhost=([A-Za-z]*-?(\\d+-\\d+-\\d+).*)",
            auth_fail_df$message[index])
p = regmatches(auth_fail_df$message[index], m)
raw_ip = pull_col(p, 2)
auth_fail_df$ip[index] = ifelse(
```

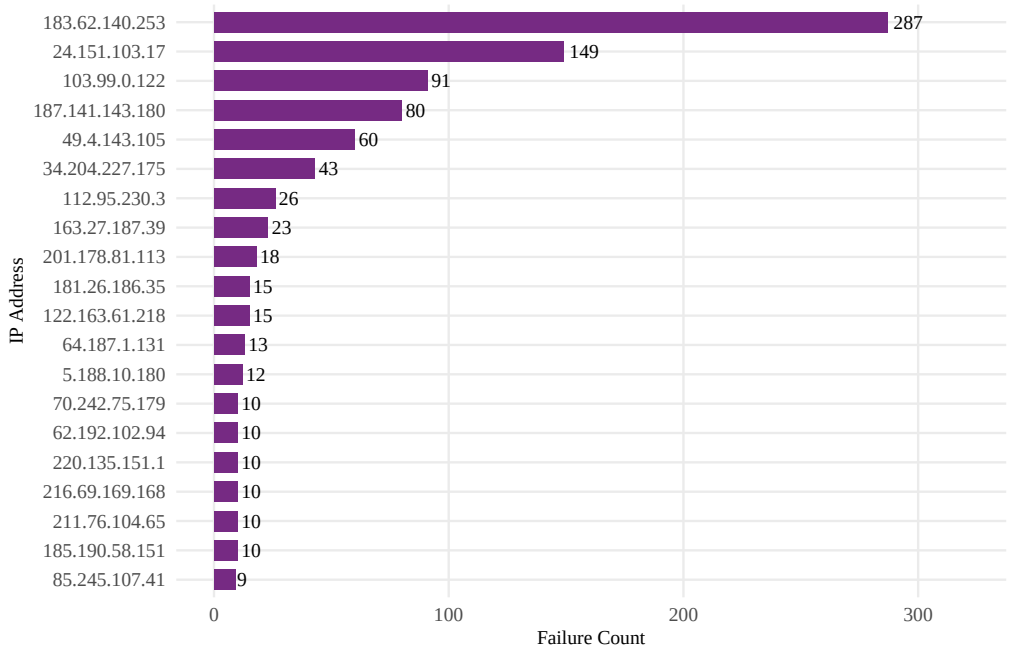
```
  grepl("^[0-9.]+$", raw_ip),
  raw_ip,
  gsub("-", ".", sub("^[A-Za-z]*-?(\\d+-\\d+-\\d+-\\d+).*", "\\1", raw_ip))
)

  # sshd: just get ip address
index = auth_fail_df$app == "sshd"
m = regexec("([0-9]{1,3}\\.[0-9]{1,3}\\.[0-9]{1,3}\\.[0-9]{1,3})",
            auth_fail_df$message[index])
p = regmatches(auth_fail_df$message[index], m)
auth_fail_df$ip[index] = pull_col(p, 2)

auth_fail_df$ip[auth_fail_df$ip == ""] = NA
```

Below features Figure 7 that showcases the top 20 IPs within authentication failures.

Figure 7: Top 20 IPs by Authentication Failures



IV. Approach of `sudo`

Using my previously written function `show_all_messages`, I applied this once again to the app `sudo` for an idea of the message patterns. To extract the user and executables from the `sudo` messages, I grabbed "COMMAND=" lines and pattern matched to get the two capture groups. To extract the machine, the logging host was used.

```
sudo_df = df[df$app == "sudo" & grepl("COMMAND=", df$message), ]
sudo_df$user = NA_character_
sudo_df$executable = NA_character_

# user is before the first :
# command path is after COMMAND=
m = regexec("^\\s*(\\S+)\\s*:.*COMMAND=([A-Za-z0-9.-_/]+\\s*.*)$", sudo_df$message)
p = regmatches(sudo_df$message, m)
sudo_df$user = pull_col(p, 2)  # e.g., ubuntu
```

```r
sudo_df$command = pull_col(p, 3)  # e.g., /usr/bin/apt-get update

exec = sub("^(\\S+)\\s*.*", "\\1", sudo_df$command)
sudo_df$executable = sub(".*/", "", exec)
sudo_table = unique(sudo_df[, c("logging_host", "user", "executable", "command")])
print(sudo_table)
```

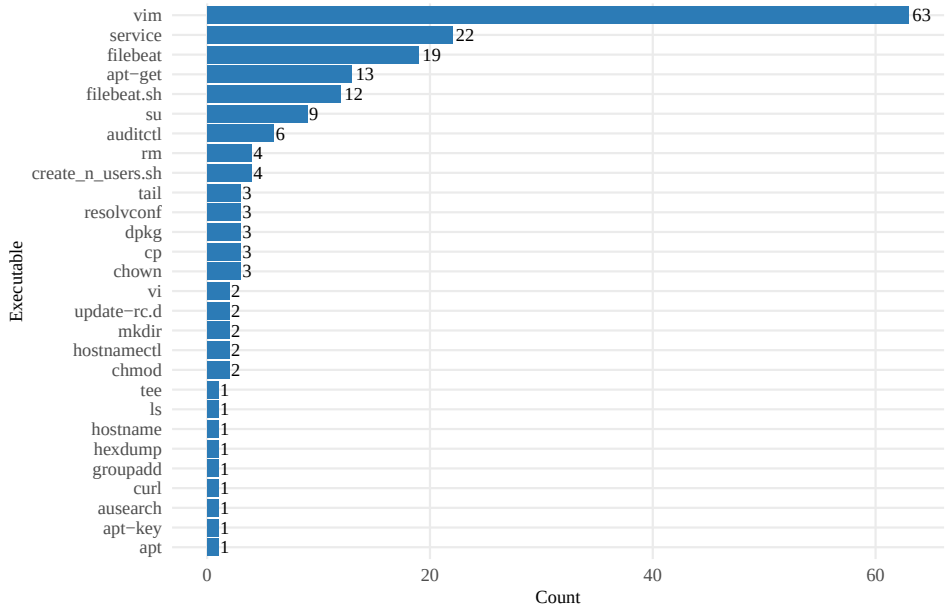
This produced a clean sudo table that consists of the logging host, user, executable, and full command line.

Table 8: Sudo Commands Preview (first 15 rows)

logging_host	user	executable	command
ip-10-77-20-248	ubuntu	curl	/usr/bin/curl -L -O https://artifacts.elastic.co/downloads/beats/...
ip-10-77-20-248	ubuntu	apt-key	/usr/bin/apt-key add -
ip-10-77-20-248	ubuntu	apt-get	/usr/bin/apt-get install apt-transport-https
ip-10-77-20-248	ubuntu	tee	/usr/bin/tee -a /etc/apt/sources.list.d/elastic-5.x.list
ip-10-77-20-248	ubuntu	apt-get	/usr/bin/apt-get update
ip-10-77-20-248	ubuntu	apt-get	/usr/bin/apt-get install filebeat
ip-10-77-20-248	ubuntu	update-rc.d	/usr/sbin/update-rc.d filebeat defaults 95 10
ip-10-77-20-248	ubuntu	apt-get	/usr/bin/apt-get install packetbeat
ip-10-77-20-248	ubuntu	update-rc.d	/usr/sbin/update-rc.d packetbeat defaults 95 10
ip-10-77-20-248	ubuntu	vim	/usr/bin/vim /etc/packetbeat/packetbeat.yml
ip-10-77-20-248	ubuntu	vim	/usr/bin/vim /etc/hostname
ip-10-77-20-248	ubuntu	vim	/usr/bin/vim /etc/hosts
ip-10-77-20-248	ubuntu	hostname	/bin/hostname ip-10-77-20-248
ip-10-77-20-248	ubuntu	hostnamectl	/usr/bin/hostnamectl ip-10-77-20-248
ip-10-77-20-248	ubuntu	hostnamectl	/usr/bin/hostnamectl set-hostname ip-10-77-20-248

Some of the unique executables can be seen in Figure 8.

Figure 8: Executables Ran via Sudo



A sanity check was initiated to ensure all executables were not NA; the result of this check was `character(0)`, proving all executables were grabbed.

V. AI Usage

The AI used was Claude. It was purposed by feeding chunks of my R file code to generate some of the visualizations, which I then tweaked to fit the format of RMarkdown PDF. It was also used sporadically to debug my main code. Refining was done by reporting my code chunk and my given error in R, working from there. Much of the tweaking in my code was trying to brainstorm patterns for `regex`, which I bounce off of Claude (but Claude did not necessarily generate initially). Claude also helped with syntax if I knew what I wanted to code but did not know the respective R function for it.

The total run time for my R file was about 11 seconds.